

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO
FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA
ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



DOCUMENTO DE ESPECIFICACIÓN Y DISEÑO ARQUITECTÓNICO (V2)

Resolución de Ejercicios Ágiles, Requerimientos IEEE 830, Modelado UML y Base de Datos Relacional

“SISTEMA INTEGRAL DE GESTIÓN DE CAPACITACIONES (SIGC-CUSCO)”

Incremento Iterativo – Versión V2.0

Asignatura: INGENIERÍA DE SOFTWARE I

Docente: Ing. Lisha Sabah Diaz Caceres

Enfoque: Desarrollo Incremental e Iterativo – Ingeniería de Requisitos

Presentado por:

- | | |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Yaranga Achahui, Aldo | 103179 |
| • Ccama Enriquez, Carolay | 210921 |

Índice

1. Resolución Formal de Ejercicios Prácticos de Flujo Ágil	1
1.1. Ejercicio 1: Una Revisión Saturada (Gestión de Flujo y Cuellos de Botella)	1
1.2. Ejercicio 2: Coordinar una Entrega (Objetivo de Ciclo y Roles)	2
2. Especificación Formal de Requerimientos de Software	3
2.1. Requerimientos Funcionales (RF) Priorizados bajo MoSCoW	4
2.2. Requerimientos No Funcionales (RNF) según ISO/IEC 25010	5
3. Modelado Visual del Sistema (UML)	5
3.1. Diagrama de Casos de Uso del Sistema (UML)	5
3.2. Diagrama de Clases del Dominio y Multiplicidades	6
4. Diseño de Base de Datos Relacional y Esquema Físico	6
4.1. Modelo Relacional y Reglas de Integridad Referencial	6
4.2. Script DDL de Implementación Física (PostgreSQL)	7
5. Arquitectura de Software y Stack Tecnológico del Proyecto	8
5.1. Capas de la Arquitectura de Software	8
5.2. Resumen de Tecnologías y Entorno de Desarrollo	9
6. Plan de Evolución Incremental e Iterativo por Sprints	10

1. Resolución Formal de Ejercicios Prácticos de Flujo Ágil

A continuación, se presentan las soluciones técnicas y metodológicas a los dos escenarios prácticos planteados en clase, aplicadas rigurosamente al contexto del proyecto SIGC-CUSCO.

1.1. Ejercicio 1: Una Revisión Saturada (Gestión de Flujo y Cuellos de Botella)

Escenario Planteado: *El límite de la columna Revisión (Review) es 1 y ya existe un ítem esperando en cola.*

1. ¿Conviene iniciar la tarea de recordatorios? Justificación:

- **Decisión Técnica:** **NO** conviene iniciarla bajo ninguna circunstancia.
- **Fundamento en la Ley de Little y Principios Kanban:** En la teoría de colas y el marco Kanban rige la Ley de Little:

$$WIP = TH \times CT \implies CT = \frac{WIP}{TH} \quad (1)$$

donde *WIP* es el trabajo en curso, *TH* es el rendimiento (*Throughput*) y *CT* es el tiempo de ciclo (*Cycle Time*). Cuando la fase de *Review* está saturada ($WIP = 1$), abrir una nueva tarea (como "Módulo de Recordatorios") eleva el *WIP* global sin incrementar la tasa de entrega (*TH*), disparando el tiempo de ciclo y generando inventario intermedio no verificado. Se aplica el principio rector: "*Stop starting, start finishing*" (Dejar de empezar y enfocarse en terminar).

2. Acuerdo de Colaboración: Estrategia de Swarming / Mob Review:

- En lugar de abrir código nuevo, el desarrollador disponible aplica **Swarming** (trabajo concentrado en equipo). Se pone a disposición del revisor para realizar pruebas cruzadas locales (*peer testing*), comprobar los criterios de aceptación y corregir observaciones de código en el acto.
- De esta manera, el ítem en revisión se aprueba y se mueve inmediatamente a **Done**, liberando la capacidad de la columna *Review* para admitir el siguiente elemento del flujo.

3. Protocolo de Actuación ante una Tarjeta Bloqueada:

- **Identificación Visual Inmediata:** La tarjeta recibe la etiqueta roja [BLOQUEADA] y se mueve al carril superior del tablero.
- **Registro de Dependencia Externa:** Se anota en la tarjeta la causa raíz específica (ej. "*Pendiente respuesta de API RENIEC para validación de DNI*" o "*Falta plantilla oficial de diploma municipal*").
- **Política de Aislamiento y SLA:** La tarjeta bloqueada no satura el límite de trabajo activo del desarrollador asignado, pero adquiere prioridad absoluta para el Facilitador/Scrum Master, con un acuerdo de nivel de servicio (SLA) de resolución inferior a 24 horas gestionado en el *Daily Standup*.

Políticas Explícitas de Flujo en el Tablero SIGC-CUSCO

- **Límites WIP Formales:** *To Do* = 5 máx. | *In Progress* = 2 máx. | *Review / QA* = 1 estricto | *Done* = Ilimitado.
- **Regla de Oro ante Bloqueos:** Si una tarjeta se bloquea por más de 12 horas, se destraba de manera colaborativa durante los primeros 5 minutos del *Daily Standup*.

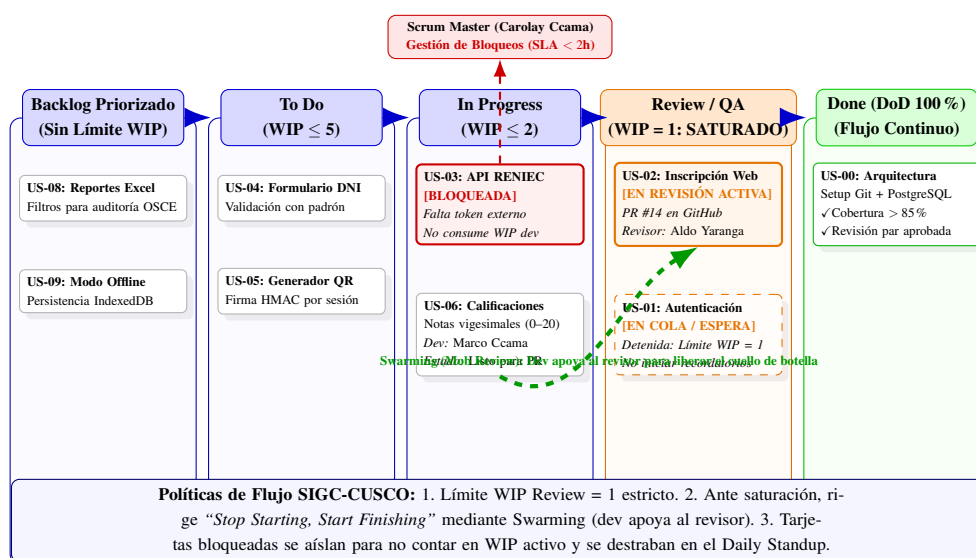


Figura 1: Visualización del flujo de trabajo Kanban, gestión del cuello de botella en Review y política de bloqueo.

1.2. Ejercicio 2: Coordinar una Entrega (Objetivo de Ciclo y Roles)

Escenario Planteado: *El equipo debe demostrar una funcionalidad utilizable en una entrega incremental.*

1. Objetivo de Ciclo Orientado a Valor de Negocio (*Sprint Goal*):

“Habilitar que un participante en Cusco complete exitosamente su inscripción digital en línea a un curso municipal o turístico mediante validación de su DNI en tiempo real, obteniendo en pantalla su credencial digital con código QR funcional para el control presencial de asistencia.”

Criterio de Valor: Representa un flujo vertical completo (*thin vertical slice*) directamente operable por el usuario final, no una mera tarea técnica horizontal (crear tablas o diseñar vistas").

2. Asignación de Roles Operativos en el Equipo:

- **Product Owner (Aldo Yaranga):** Responsable del valor de negocio. Define y prioriza los ítems del backlog según las necesidades de municipalidades y agencias turísticas de Cusco; valida el cumplimiento de los criterios de aceptación.
- **Scrum Master / Agile Facilitator (Carolay Ccama):** Responsable del flujo de trabajo y la agilidad del equipo. Vigila el respeto riguroso a los límites WIP en Trello, remueve impedimentos externos y facilita las ceremonias de revisión.

- **Tech Lead / Dev Lead (Noe Franklin Choquenaira):** Responsable de la arquitectura y la calidad del código. Diseña el esquema de datos en PostgreSQL, construye los servicios de backend, integra la seguridad criptográfica y valida las pruebas unitarias.

3. Dos Evidencias Objetivas de Aceptación (*Definition of Done - DoD*):

- **Evidencia 1 (Demostración Funcional en Vivo):** Prueba integral en navegador donde se ingresa un DNI, el sistema valida la disponibilidad de vacantes, almacena la inscripción y renderiza de forma instantánea el código QR verificable en pantalla.
- **Evidencia 2 (Garantía de Calidad de Software):** *Pull Request* en GitHub revisado y aprobado por dos integrantes del equipo, con 0 advertencias en análisis estático de código (*linter*) y script SQL ejecutado sin errores de integridad.

2. Especificación Formal de Requerimientos de Software

Bajo los estándares internacionales IEEE 830 e ISO/IEC/IEEE 29148, se formaliza la especificación técnica del sistema **SIGC-CUSCO**.

2.1. Requerimientos Funcionales (RF) Priorizados bajo MoSCoW

Código	Nombre del Requisito	Especificación Funcional Detallada	Prioridad
RF-01	Control de Acceso RBAC	Autenticación segura y autorización por roles: Administrador, Coordinador Municipal/Turístico, Docente y Participante.	Must Have
RF-02	Gestión Multientidad	Parametrización aislada de instituciones organizadoras (Municipalidades y Empresas de Turismo) con logos oficiales.	Must Have
RF-03	Catálogo de Capacitaciones	Creación de cursos con definición de fechas, horas pedagógicas, cupo máximo, docente asignado y temario.	Must Have
RF-04	Inscripción con DNI	Formulario público con validación algorítmica de DNI peruano, verificación de unicidad y control automático de vacantes.	Must Have
RF-05	Asistencia Digital QR	Registro instantáneo de asistencia mediante escaneo del código QR del participante desde el dispositivo móvil del docente.	Must Have
RF-06	Calificaciones y Actas	Registro de notas en escala vigesimal (0 a 20), cálculo automático de condición de aprobación y cierre inmutable de actas.	Should Have
RF-07	Emisión de Certificados	Generación masiva de certificados PDF firmados digitalmente, provistos de código QR y hash de autenticidad SHA-256.	Must Have
RF-08	Verificador Público Web	Portal web público donde cualquier entidad puede escanear el QR o ingresar el código para validar la autenticidad del diploma.	Must Have
RF-09	Modo Offline de Asistencia	La interfaz del docente permite registrar asistencias sin conectividad a internet, sincronizando datos al recuperar enlace.	Should Have
RF-10	Reportes de Fiscalización	Generación de padrones de asistencia y notas exportables a formato Excel (.xlsx) compatibles con auditorías de OSCE.	Could Have

Tabla 1: Matriz de Requerimientos Funcionales con priorización MoSCoW.

2.2. Requerimientos No Funcionales (RNF) según ISO/IEC 25010

Código	Dimensión de Calidad	Criterio de Aceptación Cuantitativo y Tecnológico
RNF-01	Seguridad Criptográfica	Integridad documental mediante hash SHA-256 inmutable por certificado. Almacenamiento de credenciales con cifrado adaptativo <i>bcrypt</i> ($cost \geq 12$).
RNF-02	Rendimiento en Lote	Generación asíncrona de 100 certificados PDF en un tiempo total menor a 15 segundos sin degradar el tiempo de respuesta del servidor web (< 500 ms).
RNF-03	Disponibilidad del Servicio	Arquitectura desplegada en nube con disponibilidad operativa mínima del 99.5 % en horarios hábiles de oficina.
RNF-04	Usabilidad Móvil (Responsive)	Interfaz adaptable que permite completar la inscripción ciudadana en máximo 3 pantallas y menos de 60 segundos en dispositivos móviles.
RNF-05	Tolerancia a Desconexión	Persistencia local de registros de asistencia mediante <i>IndexedDB</i> en el navegador para tolerar cortes de red en locales comunales de Cusco.

Tabla 2: Requerimientos No Funcionales clasificados según el estándar ISO/IEC 25010.

3. Modelado Visual del Sistema (UML)

3.1. Diagrama de Casos de Uso del Sistema (UML)

El modelo de casos de uso (Figura 2) define los límites del sistema **SIGC-CUSCO**, diferenciando los actores primarios de los servicios de auditoría y especificando las relaciones de inclusión ($\ll include \gg$).

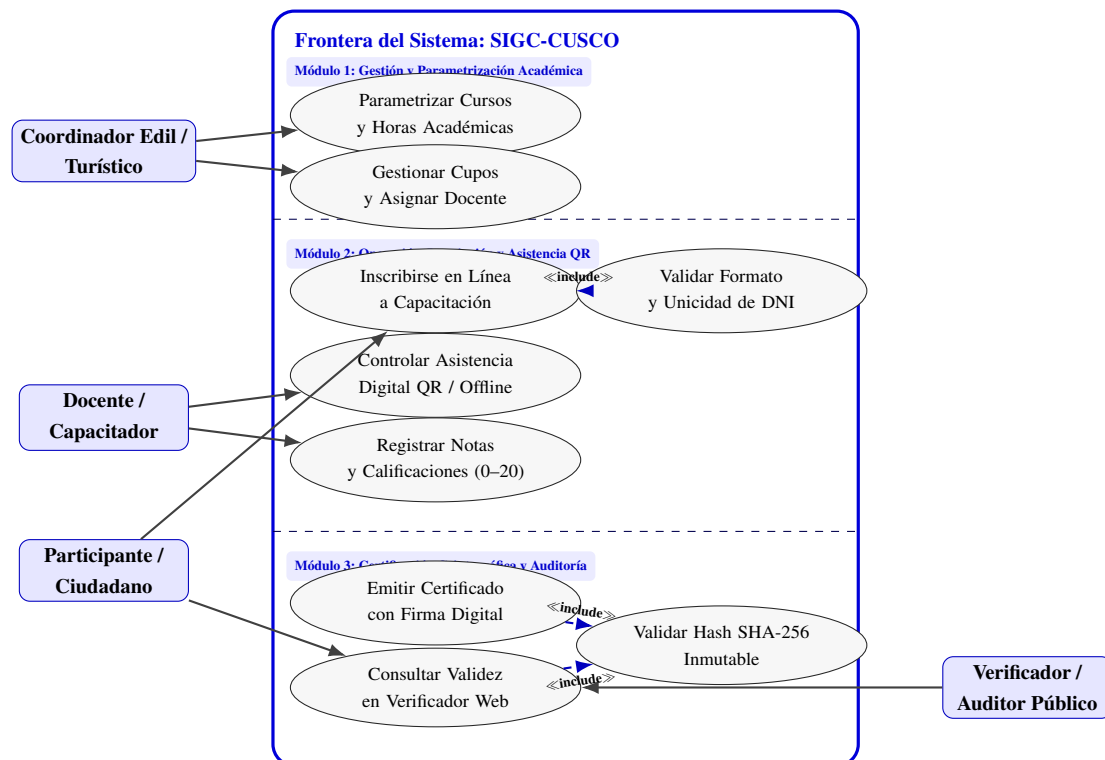


Figura 2: Diagrama de Casos de Uso del Sistema SIGC-CUSCO con módulos funcionales delimitados.

3.2. Diagrama de Clases del Dominio y Multiplicidades

El modelo de clases del dominio (Figura 3) representa las entidades estructurales del negocio y sus multiplicidades formales organizadas en una topología planar sin cruces.

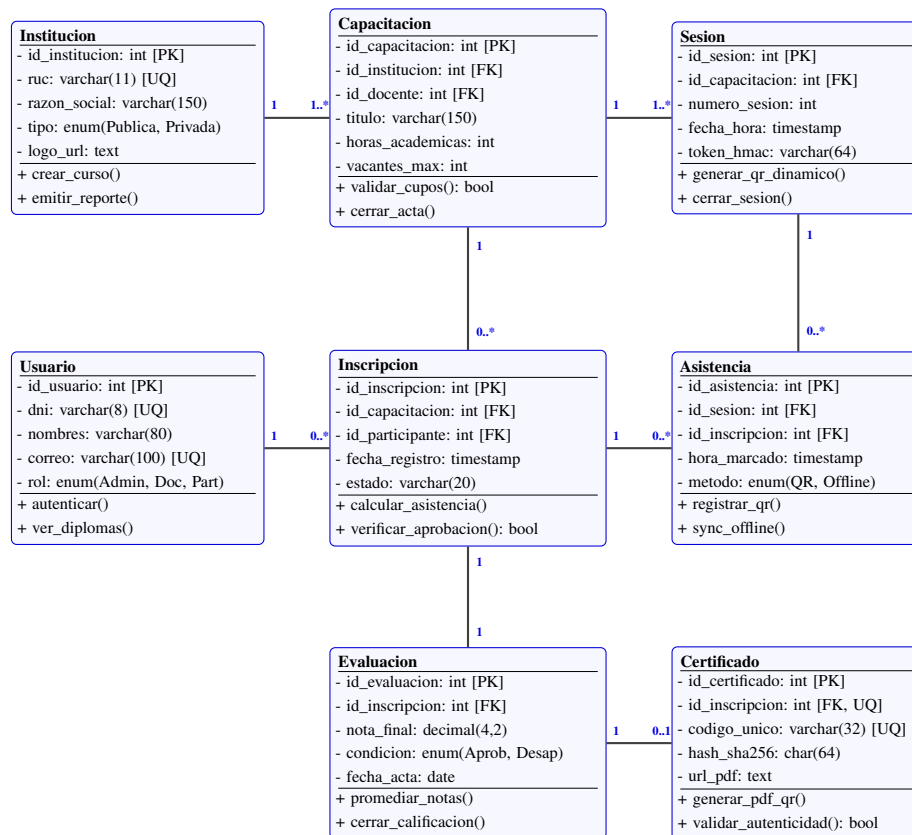


Figura 3: Diagrama de clases del dominio con 7 entidades normalizadas en grilla ortogonal, multiplicidades y trazabilidad completa.

4. Diseño de Base de Datos Relacional y Esquema Físico

El sistema implementa el gestor relacional **PostgreSQL 16** para asegurar soporte estricto de transacciones ACID y alta consistencia en la emisión documental.

4.1. Modelo Relacional y Reglas de Integridad Referencial

El diseño de datos responde a la Tercera Forma Normal (3FN), garantizando que no existan redundancias ni anomalías de actualización:

1. **Relación Uno a Muchos (1 : N) entre Institución y Capacitaciones:** Una entidad (municipalidad o empresa) patrocina múltiples cursos a lo largo del año. La eliminación de una institución se restringe si tiene cursos históricos cerrados (ON DELETE RESTRICT) para preservar la trazabilidad de auditoría.
2. **Relación Muchos a Muchos (N : M) resuelta por Entidad Asociativa:** Un participante se matricula en múltiples capacitaciones y una capacitación alberga decenas de alumnos. Esta asociación se materializa en la tabla inscripciones, la cual incluye una restricción única compuesta UNIQUE (id_capitacion, id_participante) para impedir duplicidades de matrícula.

3. **Relación Uno a Uno Condicional (1 : 0..1) con Certificados:** Una inscripción completada y aprobada genera a lo sumo un único certificado oficial. Se implementa con una restricción UNIQUE sobre la clave foránea id_inscripcion en la tabla certificados.

4.2. Script DDL de Implementación Física (PostgreSQL)

Script DDL PostgreSQL 16 – SIGC-CUSCO

```
-- 1. Entidades Organizadoras (Multitenancy)
CREATE TABLE instituciones (
    id_institucion SERIAL PRIMARY KEY,
    ruc VARCHAR(11) UNIQUE NOT NULL,
    razon_social VARCHAR(150) NOT NULL,
    tipo VARCHAR(20) NOT NULL CHECK (tipo IN ('Publica', 'Privada')),
    logo_url TEXT,
    activo BOOLEAN DEFAULT TRUE,
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Usuarios del Sistema y Roles RBAC
CREATE TABLE usuarios (
    id_usuario SERIAL PRIMARY KEY,
    dni VARCHAR(8) UNIQUE NOT NULL,
    nombres VARCHAR(80) NOT NULL,
    apellidos VARCHAR(80) NOT NULL,
    correo VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    rol VARCHAR(20) NOT NULL CHECK (rol IN ('Admin', 'Coordinador', 'Docente', 'Alumno')),
    creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Catálogo de Capacitaciones
CREATE TABLE capacitaciones (
    id_capitacion SERIAL PRIMARY KEY,
    id_institucion INT NOT NULL
        REFERENCES instituciones(id_institucion) ON DELETE RESTRICT,
    id_docente INT
        REFERENCES usuarios(id_usuario) ON DELETE SET NULL,
    titulo VARCHAR(150) NOT NULL,
    horas_academicas INT NOT NULL CHECK (horas_academicas > 0),
    vacantes_max INT NOT NULL CHECK (vacantes_max > 0),
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE,
    estado VARCHAR(20) DEFAULT 'Planificada'
        CHECK (estado IN ('Planificada', 'En_Curso', 'Finalizada'))
);

-- 4. Matrículas e Inscripciones (Tabla Intermedia)
CREATE TABLE inscripciones (
    id_inscripcion SERIAL PRIMARY KEY,
    id_capitacion INT NOT NULL
        REFERENCES capacitaciones(id_capitacion) ON DELETE CASCADE,
    id_participante INT NOT NULL
        REFERENCES usuarios(id_usuario) ON DELETE RESTRICT,
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    nota_final NUMERIC(4,2) CHECK (nota_final >= 0 AND nota_final <= 20),
    estado VARCHAR(20) DEFAULT 'Confirmada'
        CHECK (estado IN ('Confirmada', 'Aprobado', 'Desaprobado', 'Anulada')),
    CONSTRAINT uq_participante_curso UNIQUE (id_capitacion, id_participante)
);
```

```

);

-- 5. Control de Asistencias por Sesión (Soporte QR / Offline)
CREATE TABLE asistencias (
    id_asistencia SERIAL PRIMARY KEY,
    id_inscripcion INT NOT NULL
        REFERENCES inscripciones(id_inscripcion) ON DELETE CASCADE,
    numero_sesion INT NOT NULL CHECK (numero_sesion > 0),
    fecha_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    estado VARCHAR(15) NOT NULL CHECK (estado IN ('Presente', 'Tardanza', 'Falta')),
    metodo VARCHAR(15) DEFAULT 'QR_Digital'
        CHECK (metodo IN ('QR_Digital', 'PIN_Manual', 'Offline_Sync'))
);

-- 6. Emisión Criptográfica de Certificados
CREATE TABLE certificados (
    id_certificado SERIAL PRIMARY KEY,
    id_inscripcion INT UNIQUE NOT NULL
        REFERENCES inscripciones(id_inscripcion) ON DELETE RESTRICT,
    codigo_unico VARCHAR(32) UNIQUE NOT NULL,
    hash_sha256 CHAR(64) NOT NULL,
    fecha_emision TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    url_pdf TEXT NOT NULL
);

-- Índices de Alto Rendimiento para Búsqueda y Validación
CREATE INDEX idx_usuarios_dni ON usuarios(dni);
CREATE INDEX idx_inscripciones_curso ON inscripciones(id_capacitacion);
CREATE INDEX idx_certificados_hash ON certificados(hash_sha256);
CREATE INDEX idx_certificados_codigo ON certificados(codigo_unico);

```

5. Arquitectura de Software y Stack Tecnológico del Proyecto

Para garantizar robustez, mantenibilidad y escalabilidad, el proyecto **SIGC-CUSCO** adopta una arquitectura en tres capas desacopladas con componentes orientados a servicios RESTful.

5.1. Capas de la Arquitectura de Software

1. **Capa de Presentación (Frontend Web):** Desarrollada en **React 18** con empaquetador **Vite** y estilos utilitarios en **Tailwind CSS**. Ofrece una interfaz ligera, accesible desde teléfonos móviles de gama baja y optimizada para la interacción rápida de ciudadanos y docentes.
2. **Capa de Lógica de Negocio (Backend API):** Implementada con **Node.js** y **Express** bajo el estándar tipado de **TypeScript**. Gestiona la validación algorítmica de DNIs, el control concurrente de cupos, la orquestación del marcado de asistencia y la cola asíncrona de procesamiento de PDFs.
3. **Capa de Persistencia y Seguridad (Datos):** Motor **PostgreSQL 16** para el almacenamiento relacional estructurado. Generación de códigos QR mediante librerías nativas vectoriales y cálculo de resumen criptográfico inmutable mediante el algoritmo estándar SHA-256.

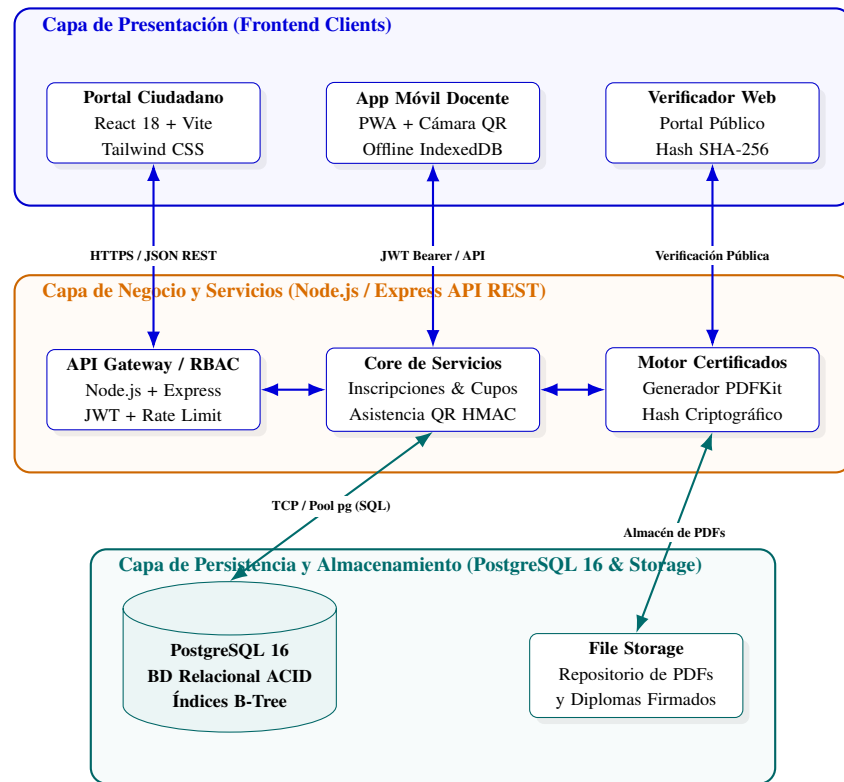


Figura 4: Arquitectura de software desacoplada en tres capas con protocolos de comunicación estandarizados.

5.2. Resumen de Tecnologías y Entorno de Desarrollo

Componente / Capa	Tecnología Elegida	Justificación Técnica en el Proyecto
Base de Datos	PostgreSQL 16	Soporte ACID estricto, integridad referencial y alto rendimiento en consultas indexadas.
Backend / API	Node.js + Express (TypeScript)	Asincronía eficiente (<i>Event Loop</i>) para procesar inscripciones masivas sin bloqueos.
Frontend Web	React + Vite + Tailwind CSS	Renderizado rápido, diseño 100 % adaptable para smartphones y soporte de PWA.
Seguridad y Hash	Algoritmo SHA-256 / Bcrypt	Certificados infalsificables auditables y protección robusta de contraseñas.
Control de Versiones	Git & GitHub (SIGC-CUSCO)	Colaboración concurrente, trazabilidad de código y despliegue continuo (CI/CD).
Gestión Ágil	Tablero Kanban en Trello	Visualización de flujo, políticas de bloqueo y control de cuellos de botella con límites WIP.

Tabla 3: Stack tecnológico formal adoptado para la construcción de SIGC-CUSCO.

6. Plan de Evolución Incremental e Iterativo por Sprints

Para cumplir con la entrega continua de valor en cada sesión del curso, el desarrollo de **SIGC-CUSCO** se planifica en cinco incrementos progresivos:

Incremento / Sprint	Entregable Técnico Principal	Criterio de Validación Objetiva
Sprint 1: Inception	Visión, Lean Canvas y Tablero Trello.	Aprobación de alcance y matriz de stakeholders en repositorio GitHub.
Sprint 2: Requisitos y Arquitectura (Actual)	Casos de uso, clases de dominio, DER y script DDL en PostgreSQL.	Compilación formal del documento V2 y ejecución limpia del esquema SQL.
Sprint 3: Core de Inscripciones	Portal público con validación de DNI y gestión de cupos en tiempo real.	Demostración de registro ciudadano y persistencia en base de datos.
Sprint 4: Asistencia Digital	Lector móvil de código QR con persistencia offline en IndexedDB.	Marcado de presencia en aula comunal y sincronización con el servidor.
Sprint 5: Certificación y Auditoría	Motor de emisión de diplomas PDF con hash SHA-256 y validador público web.	Verificación exitosa de autenticidad escaneando el código QR con cualquier celular.

Tabla 4: Roadmap de desarrollo incremental e iterativo del sistema SIGC-CUSCO.